

Report progetto finale

Gruppo 13 : Edoardo Matachione, Riccardo Mattia, Luca Medea, Luca Pietro Messenio

Sommario: Il sistema soddisfa i requisiti di progetto integrando la logica di gioco (regole complete), TUI, GUI, connessione Socket e RMI, e 2 funzionalità aggiuntive (partite multiple e database).

DIVISIONE DEL LAVORO E FLUSSO DI SVILUPPO

Il progetto è stato sviluppato in 4 fasi incremental, supportate da testing continuo.

Fase 1: Modellazione e logica di Gioco (Model server-side)

- **Attività comune:** Briefing iniziale e progettazione del modello UML.
- **Luca Medea & Luca Messenio:** Sviluppo delle carte (personaggi, eventi ed edifici) con relativi effetti.
- **Riccardo Mattia & Edoardo Matachione:** Gestione di Board, Game, Deck, Player e relativi stati della plancia. Implementazione dello state pattern per orchestrare il gioco.

Fase 2: Architettura di Rete e Comunicazione

- **Attività comune:** Progettazione dei sequence diagram (Login, Take Card, Place Totem) impostando una prima struttura per la gestione della rete.
- **Edoardo Matachione:** Setup, organizzazione del server, implementazione Socket/RMI, gestione login e disconnessione, partite multiple e setup database.
- **Riccardo Mattia & Luca Messenio:** Implementazione e gestione dei DTO e della messaggistica.

Fase 3: Interfacce Utente

- **TUI** (Riccardo Mattia): Sviluppo completo dell'architettura.
- **GUI** (Luca Messenio, Luca Medea):
 - o **Luca Medea:** Logica di comunicazione, struttura generale della GUI, e gestione flussi di comunicazione.
 - o **Luca Messenio:** Sviluppo delle componenti principali con focus su usabilità, interazione e resa grafica.

Fase 4: Refactoring

Dopo aver completato le architetture di base del progetto, sono stati effettuati dei refactoring seguendo i principi SOLID.

- Edoardo Matachione: refactoring Server
- Riccardo Mattia: refactoring TUI
- Luca Medea: refactoring GUI

Questo ha portato alla "generazione" di molte classi nel progetto che riteniamo sia una scelta giustificata per presentare un codice comprensibile e scalabile.

Alcune "criticità" emerse sono state la presenza di "god classes", interfacce troppo grandi che davano accesso a metodi non di interesse, "Single responsibility" non sempre rispettato.

Architettura e Pattern

Le scelte di design si sono orientate verso l'estendibilità e il disaccoppiamento dei componenti applicando i principi SOLID.

- **State Pattern:** Utilizzato per gestire il ciclo di vita e gli stati della partita, connessione, area di gioco e TUI.
- **Observer Pattern:** Impiegato per propagare in tempo reale gli aggiornamenti del modello verso le interfacce esterne.
- **Visitor Pattern:** Applicato per la gestione polimorfica di eventi, azioni e messaggi di rete.
- **Command Pattern:** Adottato per l'incapsulamento e l'esecuzione dei comandi da TUI.
- **Factory Pattern:** Utilizzato per la creazione centralizzata di deck ,comandi e componenti di rete.
- **Pattern MVP(GUI):** adottato per separare chiaramente Model, View e Presenter, isolando stato di gioco, rendering JavaFX e logica di interazione per migliorare modularità e testabilità.

Scelte implementative rilevanti

- **Architettura Client-Server:** Il client non modifica lo stato del gioco; invia esclusivamente richieste. Il server valida il turno, la fase e la legittimità dell'azione prima di aggiornare l'oggetto.
- **Protocolli di comunicazione:** Implementazione duale tramite Socket (con messaggi serializzati) e RMI (con invocazioni remote e callback). I due canali differiscono solo nel livello di trasporto, condividendo le medesime regole di sessione.
- **Gestione della concorrenza:** Abbiamo deciso di utilizzare istanze di "SingleThreadExecutor":
 - o Un executor per ciascuna partita attiva
 - o Un executor per ciascuna sessione utente
 - o Un executor dedicato all'output di ogni singola sessione

In particolare, ogni partita ha un executor single-thread che serializza tutte le azioni sul model. Le connessioni hanno executor separati per ordinare input e output, mentre lato client gli executor evitano di bloccare la UI e garantiscono che gli aggiornamenti vengano processati nel thread corretto.

Testing

I test sono organizzati su tre macroaree: il model, il client e il network. Per la GUI, e in parte anche con la TUI; abbiamo verificato manualmente il corretto funzionamento e rendering apportando le modifiche necessarie volta per volta.

I test del model verificano le regole di gioco vere e proprie. I test client sono concentrati soprattutto su robustezza, concorrenza e corretta gestione degli stati della TUI/model client-side. I test network sono divisi in protocollo e integrazione, controllando il corretto comportamento tra più client.

Area	Class	Method	Line	Branch
Overall Project	56%	43%	43%	32%
Server	91%	77%	82%	79%
Server Model	96%	88%	94%	88%
Client Model	100%	48%	58%	43%

Gestione e Implementazione del Database

Il database viene usato per la leaderboard. Non salviamo lo stato della partita in corso, né lobby o altro. Tutto resti in memoria sul server. Il DB viene utilizzato quando una partita finisce normalmente (se ci sono errori non si effettua nessun salvataggio e si fa rollback).

È stato utilizzato PostgreSQL tramite docker, che ci serve per rendere l'ambiente "riproducibile", così chiunque avvia il progetto può avere lo stesso PostgreSQL senza installarlo manualmente.

